

GREENOPS
IPS12 – INTELLIGENT DATA CENTER RESOURCE
OPTIMIZATION AND SUSTAINABILITY MANAGEMENT
PLATFORM

PHASE I PROJECT REPORT

Submitted by

AISHWARYA A	230701015
AISHWARYA M	230701016
KARTHIKHA SRE M	230701143
KAVIYA MADHIRAJU	230701148

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI
ANNA UNIVERSITY : CHENNAI 600 025

SEPTEMBER 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled “**IPS12 – INTELLIGENT DATA CENTER RESOURCE OPTIMIZATION AND SUSTAINABILITY MANAGEMENT PLATFORM**” is the bonafide work of AISHWARYA A (230701015), AISHWARYA M (230701016), KAVIYA MADHIRAJU (230701148), and KARTHIKHA SRE M (230701143), who carried out the project work under my supervision. The work presented in this report is prepared for academic evaluation and documents the design, implementation, machine-learning integration, telemetry pipeline, analytics dashboard, and sustainability decision-support modules of GreenOps.

This project addresses the following Sustainable Development Goal(s):

Goal No : 13 - Climate Action - Take urgent action to combat climate change and its impacts by reducing emissions and strengthening climate resilience

SIGNATURE

Dr. J. Manoranjini,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Ms. R. Akiladevi,
Supervisor,
Assistant Professor(SS),
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted for the project viva-voce examination held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Data centers consume vast electricity and water, with significant waste from idle servers, stale or duplicate storage, and unoptimized cooling. Existing sustainability tools often monitor resources in isolation and report metrics without simulating the impact of optimization actions before execution. GreenOps bridges this gap by ingesting multi-domain telemetry (compute, power, cooling, storage) to compute live metrics like PUE, WUE, energy, cost, and carbon. A statistically derived rule engine identifies idle servers and storage inefficiencies using server-type-specific thresholds. Four machine learning models support the platform: two validation models trained on 10,000 telemetry records estimate CPU utilization and power draw, while two forecasting models predict near-term workload demand. Before proposing server consolidation, GreenOps executes a what-if simulation evaluating post-move CPU, memory, network, and thermal headroom against live and forecasted limits. Recommendations are ranked by weighted sustainability and risk factors, logged for auditing, and restricted via role-based access control. By keeping final execution decisions under human operator control, GreenOps delivers transparent decision support for data center sustainability. This report details the architecture, modelling, recommendation logic, and experimental validation of the platform.

Keywords: GreenOps, Data Center Sustainability, Resource Optimization, Telemetry Analytics, Machine Learning, Workload Forecasting, What-If Simulation, PUE, WUE, Server Consolidation

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.)THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, **Ms. R. Akiladevi, M.E**, Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Mr. S.Madhusudhanan**, Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

AISHWARYA A : 230701015

AISHWARYA M : 230701016

KARTHIKHA SRE M : 230701143

KAVIYA MADHIRAJU : 230701148

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1.	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	2
	1.4 SCOPE OF THE PROJECT	3
2.	LITERATURE SURVEY	5
	2.1 LITERATURE REVIEW	5
	2.2 COMPARISON AND DISCUSSION	9
3.	SYSTEM ANALYSIS AND DESIGN	14
	3.1 OVERVIEW OF THE PROPOSED SYSTEM	14
	3.2 SYSTEM ARCHITECTURE	14
	3.3 SYSTEM DESIGN	16
	3.3.1 ACTIVITY DIAGRAM	16
	3.3.2 ER DIAGRAM	20
	3.4 DATA LAYER - TELEMETRY SIMULATION	21
	3.5 RULE-BASED WASTE DETECTION ENGINE	23
	3.6 SUSTAINABILITY METRICS	24
	3.7 MACHINE LEARNING VALIDATION MODELS	25
	3.7.1 CPU ESTIMATION MODEL	25

3.7.2	POWER ESTIMATION MODEL	25
3.7.3	MODEL SELECTION RULE	25
3.8	WORKLOAD FORECASTING	26
3.8.1	OPERATOR-FACING FORECAST	26
3.8.2	CONSOLIDATION SAFETY FORECAST	26
3.9	RECOMMENDATION ENGINE AND WHAT-IF SIMULATION	27
3.10	OPERATOR DECISIONS AND AUDIT TRAIL	29
3.11	ROLE-BASED ACCESS CONTROL	30
3.12	DATABASE DESIGN	30
4.	IMPLEMENTATION	31
4.1	TECHNOLOGY STACK	31
4.2	SYSTEM REQUIREMENTS	31
4.2.1	HARDWARE REQUIREMENTS	31
4.2.2	SOFTWARE REQUIREMENTS	32
4.3	DATABASE, REACT FRONTEND, TELEMETRY, AND API INTEGRATION	33
4.4	MACHINE-LEARNING, FORECASTING, RECOMMENDATION, AND DECISION IMPLEMENTATION	36
5.	RESULTS AND DISCUSSION	38
5.1	RULE ENGINE RESULTS	38
5.2	ML MODEL EVALUATION RESULTS	38
5.3	CONSOLIDATION WHAT-IF SIMULATION - WORKED EXAMPLE	41
5.4	HONEST SUMMARY OF SYSTEM MATURITY	41
5.4.1	GENUINELY REAL	42

5.4.2	LEGITIMATE ESTIMATES, CLEARLY LABELLED	42
5.4.3	DELIBERATELY NOT BUILT	42
5.5	SYSTEM IMPLEMENTATION	43
6.	CONCLUSION AND FUTURE WORK	48
6.1	CONCLUSION	48
6.2	LIMITATIONS	49
6.3	FUTURE WORK	49
	APPENDIX 1 - KEY GENERATING FORMULAS	51
A1.1	SERVER_MONITOR.PY	51
A1.2	POWER_MONITOR.PY	51
A1.3	COOLING_MONITOR.PY	51
A1.4	CONSOLIDATION POST-MOVE LOAD	52
A1.5	RECOMMENDATION RANKING SCORE	52
	APPENDIX 2 - DATABASE SCHEMA	53
	REFERENCES	58

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Comparative Analysis of Existing Literature	9
3.1	Telemetry Simulators and Their Output Fields	21
3.2	Simulated Server Identity and Workload Bias	21
3.3	Rule Engine Flags	23
3.4	Per-Type Idle Thresholds and Severity Weights	23
3.5	Sustainability Metrics and Trust Level	24
3.6	Flag-to-Recommendation Mapping	27
3.7	Consolidation Safety Check Dimensions	28
3.8	Role-Based Access Control Matrix	30
4.1	Primary REST Endpoint Reference	35
5.1	CPU Estimation Model - Candidate Comparison	39
5.2	Power Estimation Model - Candidate Comparison	39

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	GreenOps System Architecture	15
3.2	Activity Diagram for Monitoring and Analysis	17
3.3	Activity Diagram for Recommendation and Action	19
3.4	ER Diagram	20
5.1	Comparison of evaluated algorithms across CPU estimation, power estimation, CPU forecasting and memory forecasting	39
5.2	CPU Estimation: Predicted vs Actual	40
5.3	Dashboard Overview Page	43
5.4	Servers Details Page	43
5.5	Storage Details Page	44
5.6	Analytics Page	44
5.7	Recommendations Page	45
5.8	Approved Actions Page	45
5.9	ESG Reporting Page	46
5.10	Preferences Page	46
5.11	User Management Page	30

LIST OF ABBREVIATIONS

PUE	Power Usage Effectiveness
WUE	Water Usage Effectiveness
CPU	Central Processing Unit
kWh	Kilowatt-hour
kg	Kilogram
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
R ²	Coefficient of Determination
ML	Machine Learning
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
RBAC	Role-Based Access Control
ESG	Environmental, Social and Governance
NIC	Network Interface Card
SLA	Service Level Agreement
PDU	Power Distribution Unit
ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Data centers now account for a rapidly growing share of global electricity demand and industry studies consistently find that a significant fraction of that consumption is not doing useful work: servers sit idle at low utilization while still drawing near-full power, storage volumes accumulate stale and duplicated data that is never accessed again and cooling systems run without any real-time awareness of the load they are actually cooling. This waste is not exotic or hard to find; it is observable in ordinary infrastructure telemetry but it is rarely acted upon because the information needed to see it is scattered across separate monitoring systems for compute, power, cooling and storage and because most existing tools stop at reporting a dashboard number rather than proposing and safety-checking a concrete action.

GreenOps is a full-stack observability and recommendation platform built to address this gap directly. It structures every telemetry stream through the same five-stage pipeline: observing, analyzing, predicting, evaluating and recommending. The system never executes an infrastructure change on its own. It detects waste using transparent rule-based logic, validates current server behavior using machine learning models trained on real historical telemetry and forecasts near-term demand using models trained on the system's own accumulated history, simulates the safety and impact of a proposed consolidation before it happens and hands a ranked, itemized recommendation to a human operator who makes the final call. Every decision the operator takes is logged with the exact estimate that was shown at the time, producing an auditable trail suitable for sustainability and ESG reporting.

1.2 PROBLEM STATEMENT

Operators of data center infrastructure need a way to keep answering a few practical questions on an ongoing basis. Which servers are actually idle and worth consolidating? Which stored data is stale or duplicated and could be archived or deduplicated? Which storage volumes are over-provisioned compared to what they actually hold? And if a workload is moved onto a candidate server, is that move actually safe given the server's current and near future load, memory, network and thermal headroom? Is the data center sustainable? How much water is being used and how much carbon is being emitted? What to do to make it more sustainable?

Most existing tools don't answer all of this. They usually cover only one piece, compute or power or cooling or storage in isolation. They apply a single flat threshold no matter what type of hardware is involved and they don't simulate what would actually happen before recommending an action. Most tools don't tell us what's a directly measured value, what's a modelled estimate, and what simply isn't supported yet. That distinction matters a lot once the output is being used to justify a real infrastructure change or reported externally as a sustainability number.

1.3 OBJECTIVES

- To ingest and validate telemetry across four domains (server compute, power, cooling and storage) on a continuous, per-server basis.
- To compute live, formula-based sustainability metrics (PUE, WUE, energy, cost and carbon) and clearly separate genuinely measured ratios from labelled estimates.
- To detect operational waste, including idle servers, stale data, duplicated data and over-provisioned storage, using a rule engine with

per-server-type thresholds derived statistically from real historical data instead of one flat threshold applied to every server type.

- To train, compare and honestly report the accuracy of machine learning models that estimate CPU utilization and power draw from environmental and workload features, used as an explainable validation layer.
- To forecast near-term server workload at multiple horizons, using whichever method fits each horizon rather than forcing a single model onto every time range.
- To simulate, before any recommendation is shown, whether consolidating an idle server onto a candidate target would be safe both right now and in the near future, across CPU, memory, network and thermal dimensions.
- To rank pending recommendations using a configurable, weighted combination of estimated energy, cost, carbon and water impact against operational risk.
- To keep the human operator as the sole decision-maker: every decision and its supporting estimate is permanently logged for audit and ESG reporting and role-based access control separates who may decide from who may attest execution.

1.4 SCOPE OF THE PROJECT

The current implementation covers the complete pipeline from telemetry ingestion through to ranked, human approved recommendations and an ESG decision log. The system runs against a simulated fleet of thirteen servers spanning four server types (Compute, GPU, Storage and Edge), three cooling types (Air, Hybrid and Liquid) and three regions (APAC, EU and Middle East), with several servers in each type deliberately biased toward lower utilization so the recommendation engine always has more than one genuine candidate to work with.

Telemetry for all four domains (compute, power, cooling and storage) is produced by purpose built simulators rather than read from physical hardware, since production data center access was not available for this project. These simulators were built to mirror real monitoring behavior closely, including realistic posting intervals, range validated readings and duplicate delivery handling, so that the ingestion and analysis layers behave exactly as they would against a live facility. The two estimation models (CPU and power) are trained offline on a real, static 10,000 row “Green AI Data Center Telemetry” dataset. The two forecasting models (CPU and memory) are trained on telemetry accumulated by the live simulator itself.

The project covers rule based waste detection across all four categories (idle servers, stale data, duplicated data and over provisioned storage), with per type thresholds derived statistically. It covers a four dimension consolidation safety check spanning CPU, memory, network and thermal headroom, evaluated both against a server's current state and its near term forecast. It covers a priority weighted recommendation engine whose scoring weights an operator can adjust live and a role based access system that separates who may approve a recommendation from who may later confirm it was carried out. It also covers a dedicated ESG reporting layer that aggregates approved decisions into a facility level sustainability record, downloadable as CSV or PDF.

Cost and carbon figures throughout the system are computed using placeholder tariff and grid intensity constants, whether it is a predicted power draw, an expected saving or a water usage figure, is clearly distinguished in the interface from a value that was actually measured. That distinction was treated as a core design requirement throughout the project, since a system meant to eventually support real sustainability reporting has to be honest about which of its numbers can be trusted at face value and which are a considered estimate.

CHAPTER 2

LITERATURE SURVEY

Research on data center efficiency has developed along a few separate paths that rarely meet. One path optimizes where workloads run. Another applies machine learning to predict resource use before or after the fact. A third looks at carbon-aware scheduling and cooling. A fourth is less concerned with optimization and asks instead whether sustainability is even being measured and reported at all. The study of these existing approaches provides valuable insights into the development of the proposed GreenOps system.

2.1 LITERATURE REVIEW

The paper by Beloglazov, Abawajy and Buyya, “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” proposed heuristics for virtual machine placement and migration to improve the energy efficiency of cloud data centers. The study demonstrated that dynamic consolidation can reduce power consumption compared with static resource allocation [1]. However, the approach focuses on automated resource management and does not provide a mechanism for an operator to review a proposed migration before execution.

Subramanian, Vasan and Sivasubramaniam, in “Reducing data center power with server consolidation: Approximation and evaluation,” investigated server consolidation as an optimization problem similar to bin packing [2]. The authors developed an approximation approach for reducing data center power consumption while maintaining resource requirements. However, the approach assumes that workload demand is known and relatively stable, whereas real-world data center workloads can vary over time.

Lovász, Niedermeier and de Meer studied the performance implications of energy-aware virtual machine consolidation in “Performance tradeoffs of energy-aware virtual machine consolidation” [3]. Their work considered the tradeoff between energy savings and system performance, showing that consolidation should not be evaluated only in terms of energy reduction. Uddin et al. further investigated server consolidation and reported improvements in power efficiency and total cost of ownership [4]. However, these approaches primarily evaluate consolidation strategies rather than providing a continuous mechanism for detecting new inefficiencies.

Bawa, Kaur Chahal and Kaur proposed an ensemble learning approach for workload prediction using XGBoost and LightGBM to classify workload activity and predict CPU and memory utilization [5]. Deva and Jain investigated cloud resource utilization prediction using Azure Automated Machine Learning and XGBoost in enterprise environments [6]. These studies demonstrate the usefulness of machine learning for predicting resource requirements. However, prediction remains the primary outcome and is not directly connected to a recommendation mechanism for selecting corrective actions.

Khan, Tian, Ilager and Buyya investigated workload forecasting and energy state estimation in cloud data centers using multiple resource characteristics including provisioned resources, disk throughput and network activity [7]. Their study highlighted the importance of considering several resource parameters when estimating energy-related behaviour. The authors also explored clustering techniques for grouping machines based on energy-related characteristics. However, the approach focuses on prediction and estimation rather than extending the results into an operator-oriented recommendation and decision workflow.

Radovanović et al. presented “Carbon-Aware Computing for Datacenters,” which describes a system that shifts flexible workloads across time and geographical locations according to forecasted grid carbon intensity [8]. The study demonstrates how workload scheduling can consider carbon emissions in addition to computing requirements. However, such an approach depends on multi-region infrastructure and workload portability that may not be available in smaller or single-site data center environments.

Zhan et al. investigated data center cooling optimization using offline reinforcement learning [9]. Their approach uses historical operational data to train a reinforcement learning system without directly experimenting on a live cooling infrastructure. Alex, Ojo and Awuor proposed a carbon-aware and energy-efficient virtual machine placement approach using deep Q-networks and agglomerative clustering [10]. These studies demonstrate the potential of reinforcement learning for sustainability-oriented optimization. However, the approaches are primarily evaluated using simulation or historical data and require further validation in production environments.

Grochulska-Salak et al. argued that Power Usage Effectiveness alone is not sufficient to represent the sustainability of modern data centers [11]. Their work considers additional sustainability indicators including water usage, waste heat recovery, renewable energy integration and land use. This demonstrates the importance of considering multiple sustainability metrics rather than relying only on energy efficiency.

The European Commission has also established sustainability reporting requirements that require applicable organizations to disclose environmental and sustainability-related information [12]. Research by the Council on Energy, Environment and Water and SYSTEMIQ has examined the rapid growth of India's data center ecosystem and its associated infrastructure, energy and

resource requirements [13]. These studies highlight the increasing importance of systematic sustainability measurement as data center capacity continues to expand.

Tao and Gao examined the environmental and public-health implications associated with global data center expansion and highlighted the need for further empirical investigation into these impacts [14]. Gorey discussed the land and water implications associated with the rapid expansion of AI infrastructure and the resulting resource pressures on host regions [15]. These studies indicate that the sustainability impact of computing infrastructure extends beyond electricity consumption and includes water, land and other environmental considerations.

A 2025 Euromicro PDP study investigated sustainable data centers with emphasis on energy efficiency and resource optimization [16]. Ahmed, Brown and Gegov also investigated dynamic resource allocation through workload prediction and demonstrated how workload forecasting can support energy-efficient computing while maintaining service requirements [17]. These studies reinforce the importance of combining prediction, resource management and sustainability considerations.

From the reviewed literature, several research directions can be identified. Existing studies have addressed energy-aware resource allocation and server consolidation [1]–[4], machine-learning-based workload and resource prediction [5]–[7], carbon-aware scheduling and cooling optimization [8]–[10] and sustainability measurement and reporting [11]–[17]. However, these capabilities are generally addressed independently.

The major research gap identified from the literature is the absence of an integrated workflow combining continuous telemetry monitoring, rule-based waste detection, machine-learning-based prediction, sustainability metrics,

recommendation generation and human-reviewed decision making. Existing optimization systems may directly perform actions, prediction systems generally stop after producing forecasts and sustainability studies often focus on measurement or reporting. GreenOps addresses this gap by integrating these capabilities into a unified workflow while keeping the final operational decision with the human operator.

Thus, the reviewed literature establishes the importance of intelligent resource management, predictive modelling and comprehensive sustainability measurement in data centers. However, there remains a need for a unified system that connects these capabilities into an operational workflow. GreenOps is designed to address this gap through waste detection, prediction, sustainability assessment, recommendation and what-if simulation.

2.2 COMPARISON AND DISCUSSION

Table 2.1 Comparative Analysis of Existing Literature

Ref.	Study	ML / Optimization Technique Used	How the Model / Technique Was Used	Main Focus	Limitation / Gap Relative to GreenOps
[1]	Beloglazov et al.	Adaptive VM-consolidation heuristics	Detects overloaded and underloaded hosts, migrates VMs, consolidates workloads and powers down idle servers	Energy-aware VM allocation and server consolidation	Uses adaptive heuristics rather than workload forecasting. Does not provide broader sustainability metrics, what-if analysis or human-reviewed recommendations.
[2]	Subramanian et al.	Approximation algorithm	Models server consolidation as a variable-sized bin-packing problem to reduce the number of	Power reduction through server consolidation	Does not use ML forecasting or integrate carbon, water and other sustainability metrics with

			active servers and power consumption		operator decision support.
[3]	Lovász et al.	Energy- and performance-aware consolidation algorithms	Evaluates VM consolidation while considering both energy savings and possible performance degradation	Energy versus performance trade-off	Focuses on consolidation trade-offs rather than continuous prediction, sustainability assessment or recommendation generation.
[4]	Uddin et al.	Server-consolidation techniques	Evaluates consolidation strategies and their effects on power efficiency and total cost of ownership	Energy consumption, power efficiency and operational cost	Primarily evaluates consolidation outcomes rather than providing a continuous detection, prediction, recommendation and decision workflow.
[5]	Bawa et al.	XGBoost and LightGBM ensemble learning	Uses ensemble models to identify workload activity and predict CPU and memory utilization	Workload prediction and cloud resource management	Prediction is the main outcome. It does not connect predictions to sustainability-aware recommendations or what-if analysis.
[6]	Deva and Jain	Azure AutoML and XGBoost	Uses Azure AutoML for model selection and XGBoost for predicting cloud resource utilization	Cloud resource-utilization prediction	Focuses on prediction accuracy and does not extend the prediction into sustainability assessment, recommendations or human-reviewed decisions.
[7]	Khan et al.	Linear Regression, Ridge, ARD, ElasticNet, GRU and clustering	Uses forecasting models including GRU for workload prediction and clustering for VM	Workload forecasting and energy-state estimation	Does not integrate forecasting with waste detection, unified sustainability metrics,

			energy-state estimation		recommendations, what-if simulation and operator approval.
[8]	Radovanović et al.	Carbon-intensity forecasting, demand prediction and risk-aware optimization	Uses forecasts and optimization to shift flexible workloads toward lower-carbon periods and locations	Carbon-aware workload scheduling	Requires workload flexibility and large-scale distributed infrastructure. Does not focus on local resource waste detection or human-reviewed recommendations.
[9]	Zhan et al.	Physics-informed offline Reinforcement Learning	Learns cooling-control policies from historical operational data and applies the learned policy to optimize cooling operations	Data-center cooling optimization	Focuses specifically on cooling control and does not integrate workload forecasting, resource waste detection, broader sustainability metrics or human-reviewed recommendations.
[10]	Alex et al.	Deep Q-Network and Agglomerative Clustering	Uses clustering and deep reinforcement learning to guide carbon-aware and energy-efficient VM placement	Carbon-aware VM placement	Focuses on automated VM-placement optimization rather than an integrated monitoring, forecasting, recommendation and human-decision workflow.
[11]	Grochulska-Salak et al.	Sustainability indicators and framework	Considers energy efficiency, renewable energy, water use, waste heat, carbon emissions and land use	Holistic data-center sustainability	Provides a broad sustainability perspective but does not implement a continuous ML-based operational

					optimization system.
[12]	European Commission	Sustainability reporting framework	Establishes requirements for reporting environmental and sustainability-related information	Corporate sustainability reporting	Focuses on disclosure and reporting rather than real-time monitoring, prediction or operational recommendations.
[13]	Tripathi et al.	Industry and sustainability analysis	Examines infrastructure, energy, resource and resilience requirements associated with India's growing data-center ecosystem	Data-center infrastructure, energy and resilience	Provides ecosystem-level analysis rather than an operational system for detecting and resolving resource inefficiencies.
[14]	Tao and Gao	Environmental impact analysis	Examines environmental and public-health implications associated with data-center expansion	Environmental and societal impact of data centers	Focuses on broader environmental impacts rather than operational resource optimization or ML-based decision support.
[15]	Gorey	Environmental and resource-impact analysis	Examines land and water implications associated with the expansion of AI infrastructure	Land and water impact of AI infrastructure	Highlights resource impacts but does not provide an operational mechanism for monitoring, prediction or optimization.
[16]	Vaccaro et al.	Sustainability and resource-optimization approaches	Examines approaches for improving energy efficiency and resource utilization in sustainable data centers	Sustainable data-center operation	Provides a broad sustainability and resource-optimization perspective but does not implement a complete ML-based

					monitoring-to-decision workflow.
[17]	Ahmed et al.	Neural-network-based workload prediction	Predicts workload behaviour to support dynamic resource allocation and improve energy efficiency	Energy-efficient dynamic resource allocation	Uses prediction to support resource allocation but does not combine it with unified sustainability metrics, recommendations, what-if analysis and human approval.

From the comparative analysis, existing studies primarily focus on individual aspects of sustainable data-center management such as resource allocation, server consolidation, workload prediction, carbon-aware scheduling, cooling optimization or sustainability measurement. Although several studies employ machine learning or intelligent optimization techniques, these capabilities are generally addressed independently. Limited attention is given to integrating continuous telemetry monitoring, waste detection, workload forecasting, sustainability assessment, recommendation generation and what-if simulation into a single operator-oriented workflow. GreenOps addresses this gap by connecting these capabilities while retaining human oversight over the final operational decision.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 OVERVIEW OF THE PROPOSED SYSTEM

GreenOps watches four streams of data center telemetry, uses simple explainable rules to spot waste, uses two families of machine learning models to forecast and validate server behaviour, simulates the consequence of consolidating workloads before anything happens, and hands a human operator a ranked list of recommendations with honest, itemized impact estimates. The system never executes anything itself.

3.2 SYSTEM ARCHITECTURE

The architecture is organized into four layers: a data layer of independent telemetry simulators, a FastAPI backend that performs ingestion, storage, metrics calculation, rule-based waste detection, ML validation, forecasting, what-if simulation, and recommendation ranking; a PostgreSQL persistence layer; and a React frontend that exposes the results through role-appropriate dashboards. Figure 3.1 shows the complete architecture, from the four simulated monitors on the left, through the backend's seven internal stages, to the frontend pages on the right and the database tables at the bottom.

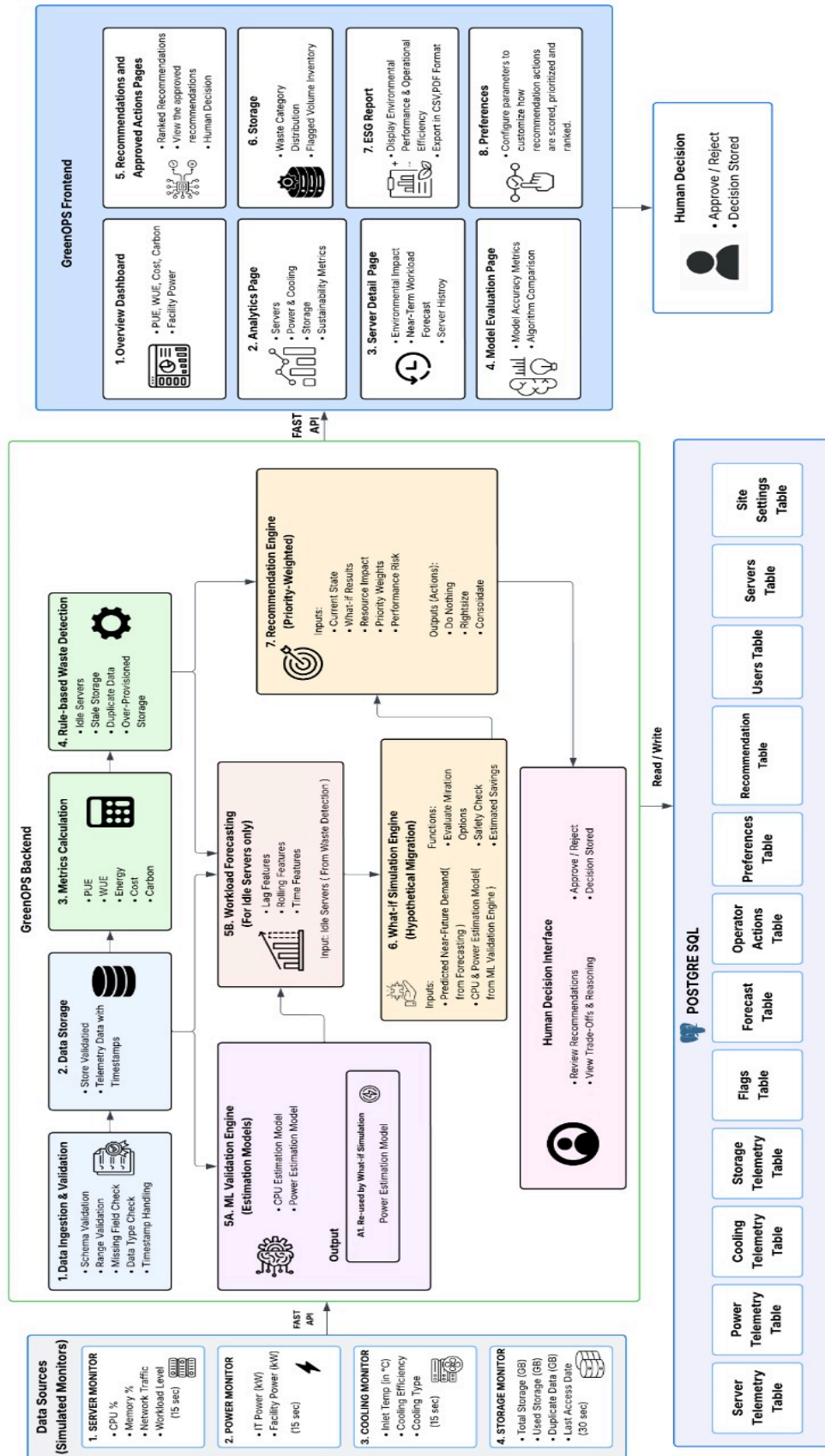


Fig 3.1 GreenOps System Architecture

At the center of the backend, the Rule-based Waste Detection module and the Metrics Calculation module both read from validated, stored telemetry and feed two parallel downstream paths: an ML Validation Engine (offline-trained CPU and power estimation models) and a Workload Forecasting module restricted to servers already flagged idle. Both paths feed a What-if Simulation Engine, which in turn feeds the priority-weighted Recommendation Engine. Every recommendation the engine produces is surfaced to a Human Decision Interface, and the operator's approve/reject/snooze decision is written back to the database, closing the loop.

3.3 SYSTEM DESIGN

3.3.1 ACTIVITY DIAGRAM

The activity diagram shown in Figure 3.2 illustrates the continuous monitoring and analysis workflow executed by the GreenOps telemetry pipeline. The process initiates with the telemetry cycle, where independent simulator scripts post structured readings every 15 to 30 seconds. Upon ingestion, the FastAPI backend performs schema validation and range checks. To ensure data integrity, any duplicate payload received within a 2-second sliding window is automatically dropped and marked as ignored.

Validated telemetry flows into the rule-based waste detection engine, which evaluates the latest observations against domain-specific thresholds for idle servers, stale data, duplicate data, and over-provisioned storage. If no waste condition is met, any previously active flag for that condition is resolved. When a condition is triggered, a new flag is raised or updated. For non-compute waste flags (archive, deduplicate, rightsize), recommendations are directly marked as pending. For idle server flags, the pipeline initiates a 15-minute workload forecast for the source server. If a safe candidate target exists and the source server is projected to remain idle, a consolidation recommendation is queued for

operator review; otherwise, the recommendation is temporarily blocked and re-evaluated during the next cycle.

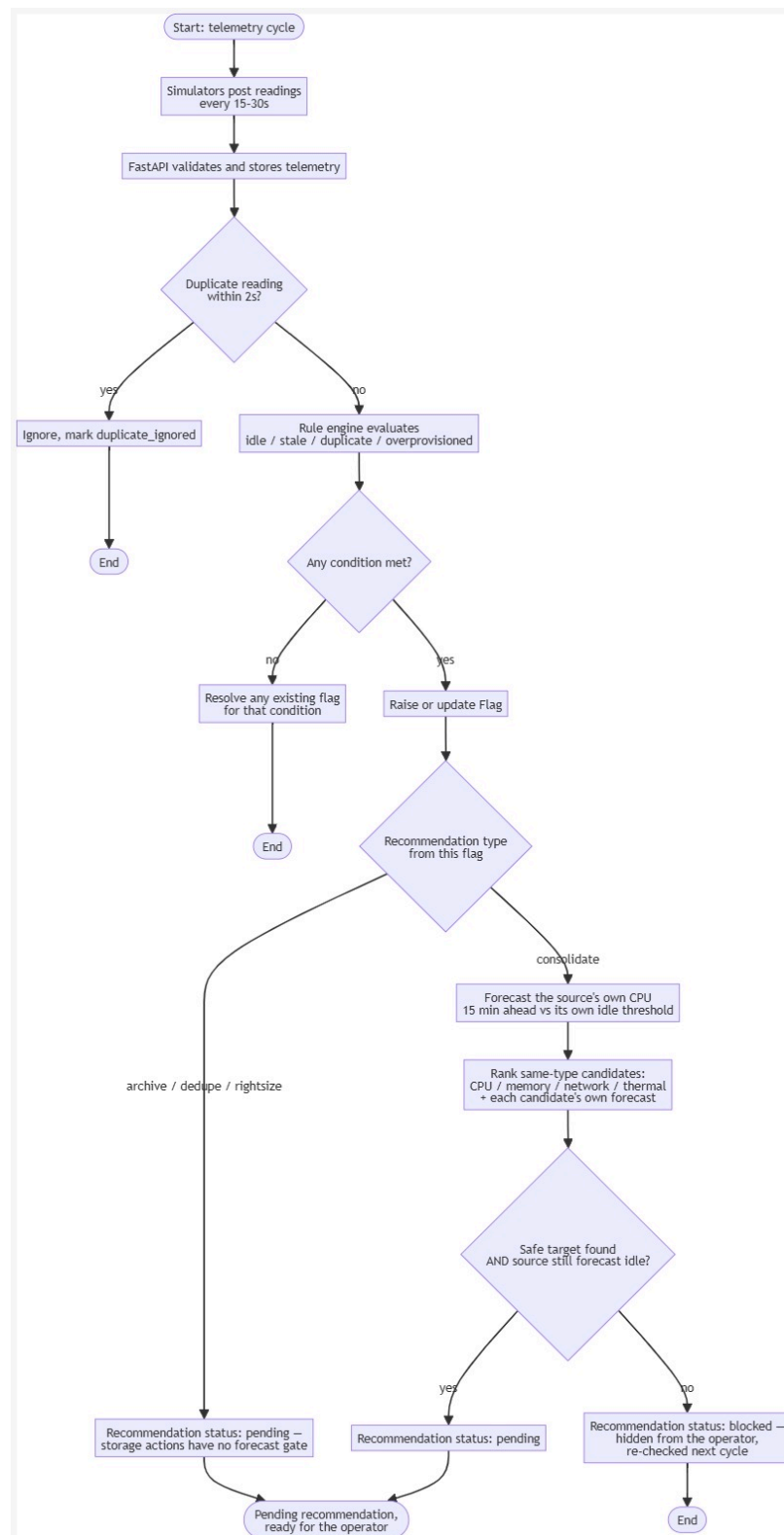


Fig 3.2 Activity Diagram for Monitoring and Analysis

Figure 3.3 details the decision-making and execution workflow for pending recommendations. Pending actions - already validated for safety in the case of server consolidations - are first prioritized and ranked by the recommendation engine using the system's weighted scoring matrix across energy, carbon, cost, water, and operational risk metrics. The ranked list is then presented to the operator through the Recommendations dashboard interface.

The human operator evaluates the presented options and selects one of three primary decision paths: Accept, Snooze, or Dismiss. Accepting a recommendation records an OperatorAction audit log entry, freezing a snapshot of the estimated savings at decision time. The accepted action can later be manually attested as executed on the Approved Actions page, allowing the ESG and compliance modules to aggregate the verified decision history. Selecting Snooze assigns a deferred timestamp, after which the recommendation automatically reverts to pending status. Dismissing a recommendation records a do-nothing action, concluding the lifecycle for that specific evaluation cycle.

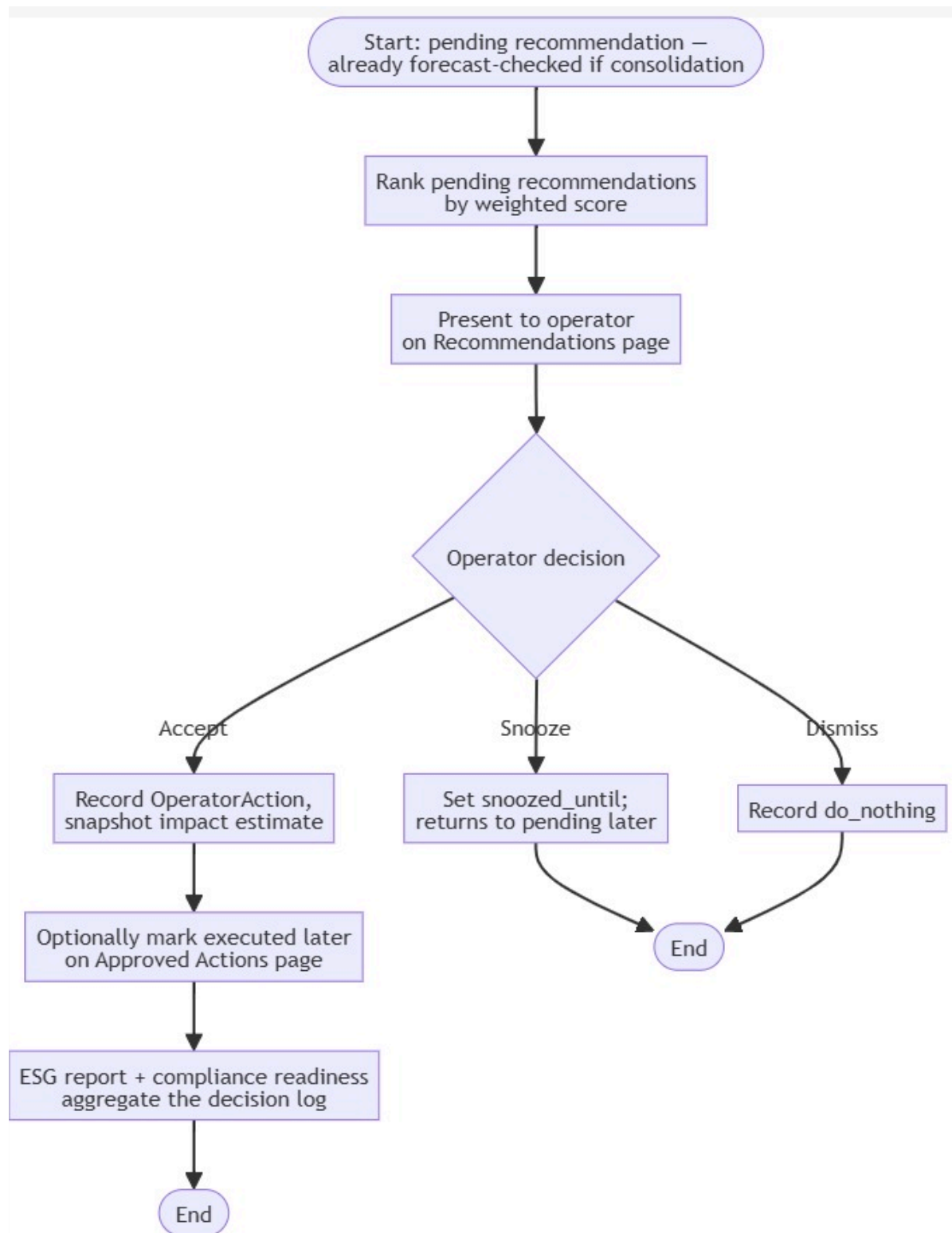


Fig 3.3 Activity Diagram for Recommendation and Action

3.3.2 ER DIAGRAM

Figure 3.4 shows the Entity-Relationship (ER) diagram for the GreenOps platform, illustrating the core database tables and their structural associations. The primary **SERVER** entity serves as the main reference for incoming telemetry streams across server, power, cooling, and storage domains. Continuously posted telemetry data triggers the rule engine to raise active **FLAG** entries, which generate prioritized **RECOMMENDATION** records evaluated by what-if simulations and site preferences. Human operators, tracked via the **USER** entity, review and decide on these recommendations, recording permanent audit entries in **OPERATOR_ACTION** along with forecasted resource usage stored in **FORECAST**.

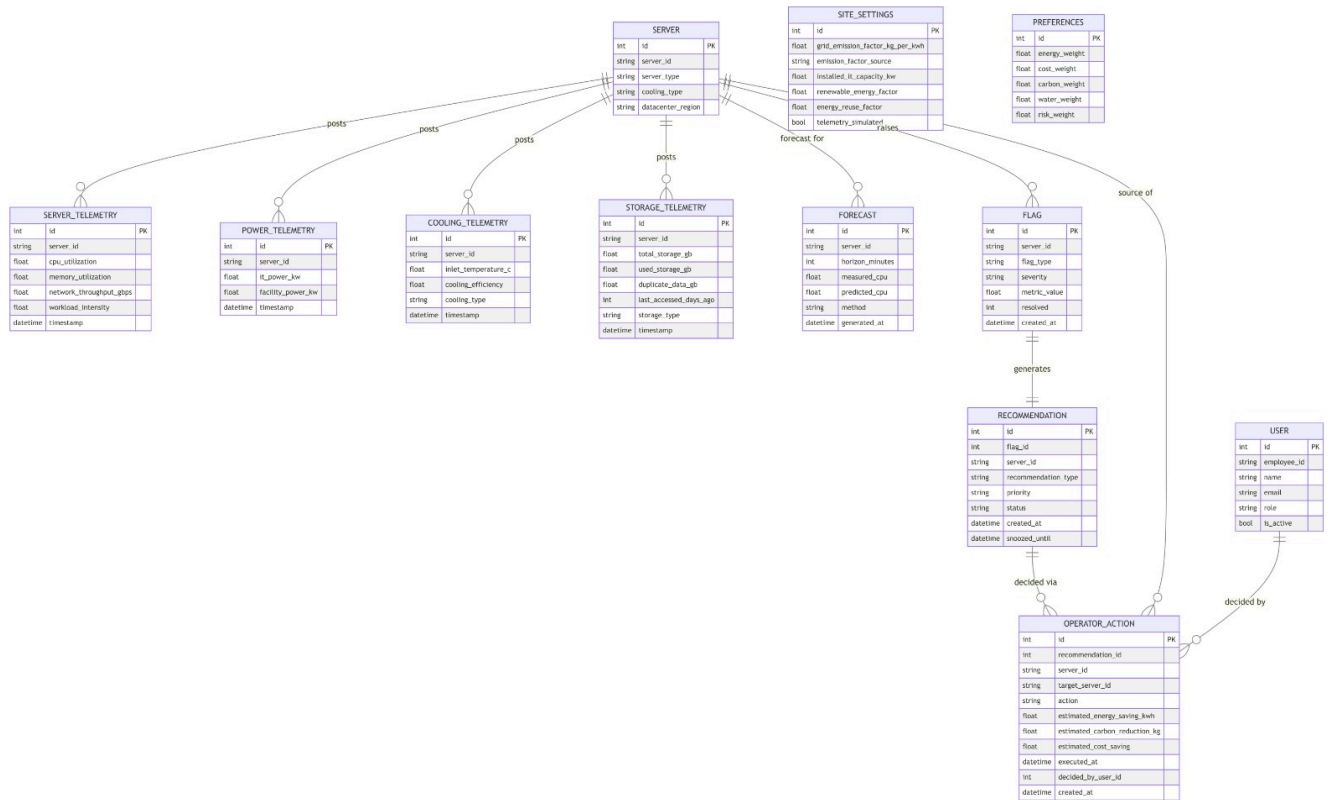


Fig 3.4 ER Diagram

3.4 DATA LAYER - TELEMETRY SIMULATION

Nothing in the running system reads real hardware. Four independent Python scripts each generate one telemetry domain and POST it to the FastAPI backend every fifteen seconds, per server, continuously, standing in for real monitoring integrations that were not available for this project.

Table 3.1 Telemetry Simulators and Their Output Fields

Simulator	Posts to	Fields
server_monitor.py	/telemetry/server	cpu_utilization, memory_utilization, network_throughput_gbps, workload_intensity
power_monitor.py	/telemetry/power	it_power_kw, facility_power_kw
cooling_monitor.py	/telemetry/cooling	inlet_temperature_c, cooling_efficiency, cooling_type
storage_monitor.py	/telemetry/storage	total_storage_gb, used_storage_gb, duplicate_data_gb, last_accessed_days_ago

Thirteen simulated servers (S1–S13) exist, each with a fixed identity assigned at registration and never changed at runtime, spanning four server types, three cooling types, and three regions, with a deliberately varied workload bias so that some servers are realistically idle and others realistically busy.

Table 3.2 Simulated Server Identity and Workload Bias

Server	Type	Cooling	Region	Workload bias
S1	Compute	Air	APAC	0.65
S2	Compute	Air	APAC	0.60
S3	GPU	Hybrid	EU	0.30
S4	Storage	Liquid	ME	0.15 (idle)
S5	Edge	Air	APAC	0.12 (idle)
S6	Compute	Air	APAC	0.05 (very idle)
S7	GPU	Hybrid	EU	0.55
S8	GPU	Hybrid	ME	0.10 (idle)

Server	Type	Cooling	Region	Workload bias
S9	Storage	Liquid	ME	0.45
S10	Storage	Liquid	APAC	0.14 (idle)
S11	Edge	Air	APAC	0.50
S12	Edge	Air	EU	0.16 (idle)
S13	Compute	Air	ME	0.70

Each simulator generates its readings from a small, explicit formula rather than pure random noise, so that later modules can be verified by hand. `server_monitor.py` draws a workload value around each server's fixed bias, and derives CPU, memory, and network from it, so memory and network carry no information CPU does not already contain. `power_monitor.py` reads each server's live CPU and memory over the API before generating its own reading, so that IT power is a real function of load, and the ratio of facility power to IT power (PUE) is a real function of the server's live cooling efficiency-both of these were previously pure random draws and were fixed during this project. `cooling_monitor.py` similarly reads each server's live CPU and applies a per-cooling-type temperature-rise and efficiency-drop curve, so that air-cooled servers heat up fastest and lose efficiency fastest under load, hybrid sits between, and liquid barely moves; this causal direction runs only from load to environment, never the reverse, which is a known and explicitly documented limitation. `storage_monitor.py` generates capacity, staleness, and duplication metadata.

FastAPI validates every incoming reading against a Pydantic schema, writes it to the database with a timestamp, and silently drops an exact-duplicate delivery from the same server arriving within two seconds, so that a network retry never double-counts a reading.

3.5 RULE-BASED WASTE DETECTION ENGINE

Four checks, each simple enough to verify by hand, run continuously against the latest stored telemetry.

Table 3.3 Rule Engine Flags

Flag	Rule
idle_server	6-hour average CPU below that server type's threshold
stale_data	last_accessed_days_ago > 90
duplicate_data	duplicate_data_gb ÷ total_storage_gb > 20%
overprovisioned	used_storage_gb ÷ total_storage_gb < 30%

The idle threshold is not a single flat number applied to every server. Per-type thresholds are derived once from the real “**Green AI Data Center Telemetry**” dataset as

$$\text{threshold} = \text{mean_cpu}(\text{type}) - 1.5 * \text{std_cpu}(\text{type})$$

clamped to the range [5%, 35%], with a severity weight scaled to that type's average power draw relative to the lowest-power type. This means an idle GPU server, which draws roughly 66% more power on average than an idle Edge server at the same CPU percentage in the real dataset, is flagged as materially higher-priority waste rather than identically to any other idle server.

Table 3.4 Per-Type Idle Thresholds and Severity Weights

Server type	Idle threshold	Severity weight
GPU	16.9%	1.66×
Compute	17.3%	1.22×
Storage	16.8%	1.06×
Edge	18.9%	1.0× (baseline)

A flag that stops being true is resolved on the same cadence it was raised, so nothing accumulates indefinitely; the synchronization routine also withdraws any pending recommendation whose underlying flag is no longer active, so a server flapping between idle and healthy does not pile up stale duplicate recommendations.

3.6 SUSTAINABILITY METRICS

Table 3.5 lists every sustainability metric the system computes, together with an honest statement of how trustworthy each one is—a distinction the system is deliberately explicit about throughout, rather than presenting every number with equal apparent confidence.

Table 3.5 Sustainability Metrics and Trust Level

Metric	Formula	Trust level
PUE	$\text{facility_power_kw} \div \text{it_power_kw}$	Genuine ratio of two directly simulated values
Energy (kWh)	$\Sigma (\text{Power} * \text{Elapsed Time})$	Corrected; previously over-counted $\approx 4\times$ by assuming one reading per minute
Cost	$\text{energy_kwh} \times \text{Tariff}$	Estimate
Carbon	$\text{energy_kwh} \times \text{Carbon Factor}$	Estimate
Total water (L)	Cumulative $\text{energy_kwh} \times \text{WUE factor}$ by cooling type, since telemetry began	Estimate, running total
WUE (L/kWh)	$\text{Water Consumption} \div \text{Energy}$	Estimate, genuine ratio in shape

Water and carbon figures use published industry-average placeholders (WUE factors of 0.3, 1.8, and 0.9 L/kWh for Air, Hybrid, and Liquid cooling respectively) keyed only by the categorical cooling type, never by the live numeric cooling-efficiency reading. In a production deployment, cost, carbon, and WUE would be replaced by a real electricity tariff, a live grid carbon-intensity feed, and a real facility water meter.

3.7 MACHINE LEARNING VALIDATION MODELS

Two regression models are trained once, offline, on a real, static, 10,000-row Kaggle dataset (“Green AI Data Center Telemetry”)-never on live simulator output, so the two pipelines share only a schema, never an origin. Shared preprocessing scales numeric features and one-hot-encodes categorical features with the first category dropped, which is necessary because, without dropping it, one-hot-encoding every category alongside an intercept term creates a perfectly collinear feature matrix, a bug that was found and fixed during development.

3.7.1 CPU ESTIMATION MODEL

Target: `cpu_utilization`.

Inputs: `workload_intensity`, `memory_utilization`, `network_throughput_gbps`, `inlet_temperature_c`, `cooling_efficiency`, `pue`, plus the categorical features `server_type`, `cooling_type`, `datacenter_region`, and `time_of_day`.

The model deliberately excludes its own target from its inputs to avoid leakage.

3.7.2 POWER ESTIMATION MODEL

Target: power, rescaled from the dataset's 28–233 kW range down to the live simulated 1.2–4.5 kW range-a pure unit change, confirmed by identical R^2 before and after rescaling.

Inputs are the same six numeric and four categorical features as the CPU model, plus `cpu_utilization`, which is legitimate here since CPU is not the target for this model.

3.7.3 MODEL SELECTION RULE

Three candidate algorithms-Linear Regression, Random Forest (200 estimators), and XGBoost (200 estimators, depth 5) are trained for each target on an 80/20 split. The lowest mean absolute error wins by default, but a more complex

model (Random Forest or XGBoost) must beat Linear Regression by at least 5% relative MAE improvement to be selected over it, a promotion bar that was fixed before results were seen. On this dataset, Linear Regression won both targets detailed results are reported in Chapter 5.

3.8 WORKLOAD FORECASTING

The forecasting module answers one question per tier: given a server's history, what will its CPU likely be? Two structurally different forecasting jobs exist and are not interchangeable.

3.8.1 *OPERATOR-FACING FORECAST*

Gated to servers currently flagged idle, this forecast covers three horizons, each handled by the method appropriate to its range rather than one method forced onto all three. The one-hour horizon uses its own trained lagged-feature model (five-, ten-, fifteen-, and twenty-minute CPU lags, the rolling one-hour average, memory, workload intensity, network throughput, and time-of-day encoded as sine/cosine), falling back to short-horizon trend extrapolation until at least thirty real lagged samples exist. The six-hour and twenty-four-hour horizons deliberately use no trained model at all: there is not yet enough real history at that range, and because each server's workload is generated from a fixed bias that never drifts or cycles, the honest estimate that far out is reversion to the server's own recent average rather than a straight-line extrapolation, which would badly overshoot. The long-horizon forecast explicitly refuses to answer - returning “**not enough history**” rather than a guess-unless real telemetry coverage reaches at least half the requested window with at least ten samples.

3.8.2 *CONSOLIDATION SAFETY FORECAST*

A completely separate, fifteen-minute-horizon model, used internally only by the what-if simulation described in Section 3.8, and always run on any

registered server with no idle gate-since a consolidation target is, by definition, not idle, and could never pass an idle-only gate. This forecast reuses the same CPU-lag architecture as the operator-facing model, plus a second, independently trained memory forecast model, since the safety check needs both dimensions.

Live prediction errors for any of these models will typically run higher than the reported offline metrics, because the models were trained on real historical relationships while live data comes from simulators using simpler generating formulas-a genuine, expected train/inference gap rather than a defect.

3.9 RECOMMENDATION ENGINE AND WHAT-IF SIMULATION

Each active flag is mapped to a recommended action type.

Table 3.6 Flag-to-Recommendation Mapping

Flag	Recommended action
idle_server	consolidate
stale_data	archive
duplicate_data	deduplicate
overprovisioned	rightsize

For idle_server flags specifically, an actual what-if simulation is run rather than a source-only guess. Every same-type server that is not itself idle-flagged is considered a candidate. For each candidate, post-move CPU, memory, and network are computed by adding the source server's own load, discounted by a 0.9 overhead factor, onto the candidate's current load. Each candidate is then checked across four safety dimensions, all of which must pass for the candidate to be considered safe right now.

Table 3.7 Consolidation Safety Check Dimensions

Dimension	Limit	Basis
CPU	< 75%	Safety limit constant
Memory	< 75%	Safety limit constant
Network	< 18.75 Gbps	75% of an assumed 25 Gbps NIC
Thermal	Predicted post-move inlet temp < 30°C	Reuses the cooling simulator's own load→temperature formula

Independently of the current-state check, the candidate's own near-term CPU and memory forecast (15 minutes ahead, Section 3.7.2) is checked against the same limits. A candidate only counts as fully safe when both the current-state check and the forecast check pass - this catches a target that looks fine right now but is about to get busy on its own. Network and thermal have no forecast counterpart, since no trained model exists for either, so they remain current-snapshot checks only. The top-ranked safe candidate (safe first, then most remaining headroom) becomes the recommended target; if no candidate qualifies, the recommendation is explicitly marked unsafe with a stated reason, never silently approved, and all considered candidates (up to five) are exposed in the response so an operator can see what else was evaluated. The impact estimate for a recommended consolidation prefers the trained power model's prediction for the target's full post-move profile, scaled to facility power by the target's current PUE, falling back to a simpler CPU-ratio formula only if the model or the target's cooling telemetry is unavailable; every use of the power model in an estimate is stated explicitly, including the caveat that it is a legitimate estimate rather than a measurement.

Pending recommendations across all flag types are ranked by:

$$\text{score} = \Sigma(\text{weight}_i \times \text{normalized_impact}_i) - \text{risk_weight} * \text{risk}$$

normalized against the maximum of each dimension within the current pending batch so that kWh, currency, kilograms, and litres do not dominate each other purely by raw magnitude. The weights are stored in a single-row, organization-configurable Preferences table editable from the frontend; changing them re-ranks recommendations instantly without retraining anything. Risk for a consolidation recommendation is

$$\text{max(post_move_cpu, post_move_memory)} \div 75$$

for storage-only actions, which have no target server and no power effect, risk is a fixed 0.1.

3.10 OPERATOR DECISIONS AND AUDIT TRAIL

An operator with the `infrastructure_manager` role can consolidate (or the generic `rightsize` verb, for archive/deduplicate actions), snooze, or dismiss any pending recommendation. Every decision snapshots the what-if impact exactly as it was shown at the moment of the decision into an `OperatorAction` record-the ESG report reads this frozen snapshot, not a live recalculation, so the log always reflects exactly what the operator actually saw when deciding. Snoozing sets a `snoozed_until` timestamp; once that passes, the recommendation quietly returns to pending rather than being lost.

The `Approved Actions` page is the one narrow exception to the rule that GreenOps has no way to verify anything actually happened: an operator can manually attest, after the fact, that a consolidation was carried out. This is attestation, not verification-there is still no automatic confirmation mechanism anywhere in the system-and the ESG report that aggregates these records is explicitly a decision log, not a verified-savings report.

3.11 ROLE-BASED ACCESS CONTROL

Three roles are enforced both server-side, via a dependency checked on every mutating endpoint, and client-side, via route guards and conditional navigation.

Table 3.8 Role-Based Access Control Matrix

Role	Can do
infrastructure_manager	Everything-accept/snooze/dismiss recommendations, edit preferences, manage users
sustainability_manager	Read-only-dashboards and the ESG report
operations_engineer	Mark approved actions as executed; cannot make the original accept/snooze/dismiss decision

Authentication is JWT-based, with users seeded through a dedicated script.

3.12 DATABASE DESIGN

PostgreSQL is used in deployment, with SQLite available as a zero-code-change default controlled by a single environment variable. Schema changes are managed through Alembic, baselined onto the pre-existing schema. Eleven tables make up the schema: **users, servers (static metadata), four raw telemetry tables (server_telemetry, power_telemetry, cooling_telemetry, storage_telemetry), forecasts (a persisted forecast audit log), flags, recommendations, operator_actions, and preferences.**

CHAPTER 4

IMPLEMENTATION

4.1 TECHNOLOGY STACK

- **Backend:** Python, FastAPI, Pydantic (validation), SQLAlchemy (ORM), Alembic (migrations).
- **Database:** PostgreSQL in deployment, SQLite for local development, selected via the DATABASE_URL environment variable.
- **Machine learning:** scikit-learn (Linear Regression, Random Forest, preprocessing pipeline), XGBoost.
- **Frontend:** React, Vite, react-router-dom, recharts.
- **Authentication:** JSON Web Tokens (JWT).
- **Simulation:** independent Python scripts per telemetry domain, run as long-lived processes..

4.2 SYSTEM REQUIREMENTS

4.2.1 HARDWARE REQUIREMENTS

Component	Minimum Recommendation	Purpose
Processor	Dual-core 2 GHz or better	Runs the backend, frontend, and simulators.
Memory	8 GB RAM	Supports PostgreSQL, Python packages, browser, and model training.
Storage	At least 5 GB free space	Stores source code, dependencies, database, logs, and model artifacts.
Network	Localhost network access	Allows frontend, backend,

		and simulator processes to communicate.
Display	1280 × 720 or better	Recommended for dashboard and analytics review.

4.2.2 SOFTWARE REQUIREMENTS

Software	Recommended role
Python 3.10 or later	Backend, simulators, training, and tests.
Node.js and npm	React/Vite frontend development.
PostgreSQL 14 or later	Production-style relational database.
Git	Source-code version control.
Visual Studio Code	Local development environment.
Modern web browser	Dashboard access and UI testing.

The project is installed locally using the following commands:

```
None
cd greenops
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r backend/requirements.txt
cd frontend
npm install
```

On Windows PowerShell, the Python environment is activated using:

```
None
.venv\Scripts\Activate.ps1
```

4.3 DATABASE, REACT FRONTEND, TELEMETRY, AND API INTEGRATION

DATABASE CONFIGURATION

PostgreSQL is created using the following SQL commands:

SQL

```
CREATE DATABASE greenops; CREATE USER greenops_user WITH PASSWORD 'change_this_password';  
GRANT ALL PRIVILEGES ON DATABASE greenops TO greenops_user;
```

The backend reads the database connection from an environment variable.

SQL

```
DATABASE_URL="postgresql+psycpg2://greenops_user:change_this_password@localhost:5432/greenops"
```

The main database tables store users, server metadata, server telemetry, power telemetry, cooling telemetry, storage telemetry, forecasts, flags, recommendations, operator actions, and priority preferences. Raw telemetry and decision records are stored in the database, while serialized machine-learning model files are stored as artifacts rather than as binary database columns.

SQLAlchemy maps Python classes to database tables. A telemetry record is first validated by Pydantic, then converted into a SQLAlchemy object, added to the active session, committed, and returned as a JSON response. Alembic is used to manage controlled schema changes when the database structure evolves.

REACT AND VITE FRONTEND

The React interface is built with Vite. Vite provides the development server, module bundling, fast refresh, and production build process. In the existing project, the frontend is already initialized; its dependencies are installed with:

None

```
cd frontend  
npm install
```

```
//The frontend development server is started using:  
npm run dev
```

The frontend displays values supplied by the backend rather than independently recalculating authoritative metrics. This keeps the backend as the source of truth and avoids inconsistent values between screens. Recharts is used to display time-series trends, server comparisons, utilization changes, energy patterns, and forecast results.

TELEMETRY SIMULATOR EXECUTION

Four independent Python processes generate the telemetry used during the demonstration. They are started in separate terminals after the backend is available.

```
None  
python simulators/server_monitor.py  
python simulators/power_monitor.py  
python simulators/cooling_monitor.py  
python simulators/storage_monitor.py
```

API INTEGRATION

The API is divided into functional groups. Telemetry endpoints receive records from the simulators. Server endpoints return current state and historical records. Analytics endpoints calculate time-window summaries. Forecast endpoints return future workload estimates. Recommendation endpoints return ranked actions and what-if details. Operator-action endpoints store human decisions. Report endpoints return ESG and decision-log information.

Table 4.1 Primary REST Endpoint Reference

Endpoint	Purpose
POST /telemetry/server	Ingest server telemetry
POST /telemetry/power	Ingest power telemetry
POST /telemetry/cooling	Ingest cooling telemetry
POST /telemetry/storage	Ingest storage telemetry
POST /servers/register	Register a new server before it can post telemetry
GET /servers	Current state of all servers
GET /servers/{id}/detail	Combined live view for one server, used by the Server Detail page
GET /servers/{id}/history	Historical telemetry for one server
GET /servers/{id}/prediction	ML CPU and power estimate vs measured
GET /servers/{id}/forecast	One-hour, six-hour or twenty-four-hour CPU projection
GET /forecasts	Persisted forecast history and validation metrics
GET /flags	All active rule-based flags
GET /storage/flags	Storage-specific flags only
GET /thresholds	Per-server-type idle thresholds and severity weights
GET /analytics/summary	PUE, WUE, energy, cost and carbon overview
GET /analytics/daily	Daily time-series data behind the Analytics page's charts
GET /analytics/history	Longer-range historical analytics data
GET /recommendations	Ranked pending recommendations
GET /recommendations/{id}	Full detail for one recommendation, without re-running the simulation
GET /recommendations/{id}/what-if	Consolidation safety and impact simulation
POST /recommendations/{id}/action	Record the operator's accept, snooze or dismiss decision
GET /operator-actions	Decision history for the Approved Actions page
PATCH /operator-actions/{id}	Attest that an approved action was executed
GET /preferences	Read the current recommendation-ranking weights

Endpoint	Purpose
PUT /preferences	Update the recommendation-ranking weights
GET /reports/esg	Aggregated ESG summary over a date range
GET /reports/esg/export	ESG report as CSV
GET /reports/esg/export/pdf	ESG report as PDF
GET /model/evaluation	Three-model comparison metrics for CPU and power estimation
POST /auth/login	Authenticate and receive a session token
GET /auth/me	Return the currently authenticated user's role and details
GET /users, POST /users, PATCH /users/{id}, DELETE /users/{id}	Manage user accounts and roles, infrastructure manager only

4.4 MACHINE-LEARNING, FORECASTING, RECOMMENDATION, AND DECISION IMPLEMENTATION

The machine-learning layer contains two related but separate tasks. The CPU estimation model estimates the current CPU utilization from current workload and environmental features. The workload-forecasting model predicts a future CPU or workload value using timestamped lag features. The CPU model is not silently renamed as the forecasting model; the two tasks have different targets and input structures.

The offline training process loads the historical telemetry dataset, handles missing values, encodes categorical features, scales numeric features where required, constructs the training matrix, and splits the data chronologically. Linear Regression, Random Forest, and XGBoost are compared using MAE, RMSE, and R^2 . The selected model and its evaluation metrics are saved for later inference.

A forecast generated at 10:00 for 10:15 is not considered validated immediately. When telemetry for 10:15 becomes available, the predicted value is compared

with the actual value. The system can then calculate the absolute error, squared error, and aggregate MAE or RMSE over many forecasts.

The recommendation engine begins with active flags. An idle-server flag may produce a consolidation recommendation, stale data may produce an archive recommendation, duplicate data may produce a deduplication recommendation, and over-provisioned storage may produce a rightsizing recommendation. For consolidation, the what-if engine evaluates candidate target servers by adding the source workload with an overhead factor. It checks CPU, memory, network, and thermal safety limits in both the current state and the near-term forecast state.

The recommendation score combines normalized energy, cost, carbon, water, and risk impacts using configurable weights. The recommended action is not automatically executed. The human decision interface allows an authorized operator to approve, snooze, dismiss, or reject the recommendation. The system stores the selected decision, user identity, timestamp, score, candidate details, and impact snapshot in an audit record.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 RULE ENGINE RESULTS

With the thirteen simulated servers running continuously, the rule engine correctly identifies the servers deliberately biased toward idle (S4, S5, S6, S8, S10, S12 in Table 3.1) within the first few minutes of telemetry collection and correctly assigns the higher severity weight to idle GPU servers (S8) relative to idle servers of other types at a comparable CPU percentage, matching the per-type thresholds derived in Section 3.4. Flags resolve automatically once a server's rolling average CPU rises back above its type's threshold and no duplicate recommendation accumulates when a server flaps between idle and healthy across consecutive evaluation windows, confirming the intended behaviour of the flag-synchronization routine.

5.2 ML MODEL EVALUATION RESULTS

Both estimation models were trained on the real, 10,000-row “Green AI Data Center Telemetry” dataset (15 columns, with missing values only in `datacenter_region`, imputed with the most frequent category inside the preprocessing pipeline). Tables 5.1 and 5.2 report every candidate algorithm's result. In both cases Linear Regression was selected since neither Random Forest nor XGBoost cleared the pre-registered 5% relative MAE improvement bar over it. Figure 5.1 compares the MAE values across the evaluated models and provides a visual comparison of the model performance.

Table 5.1 CPU Estimation Model: Candidate Comparison
(Target: cpu_utilization)

Model	MAE	RMSE	R ²
Linear Regression (selected)	4.511	7.671	0.845
Random Forest	4.828	8.077	0.828
XGBoost	4.833	7.999	0.831

Table 5.2 Power Estimation Model: Candidate Comparison
(Target: power, rescaled to 1.2–4.5 kW)

Model	MAE	RMSE	R ²
Linear Regression (selected)	0.211	0.330	0.422
Random Forest	0.227	0.346	0.366
XGBoost	0.222	0.341	0.385

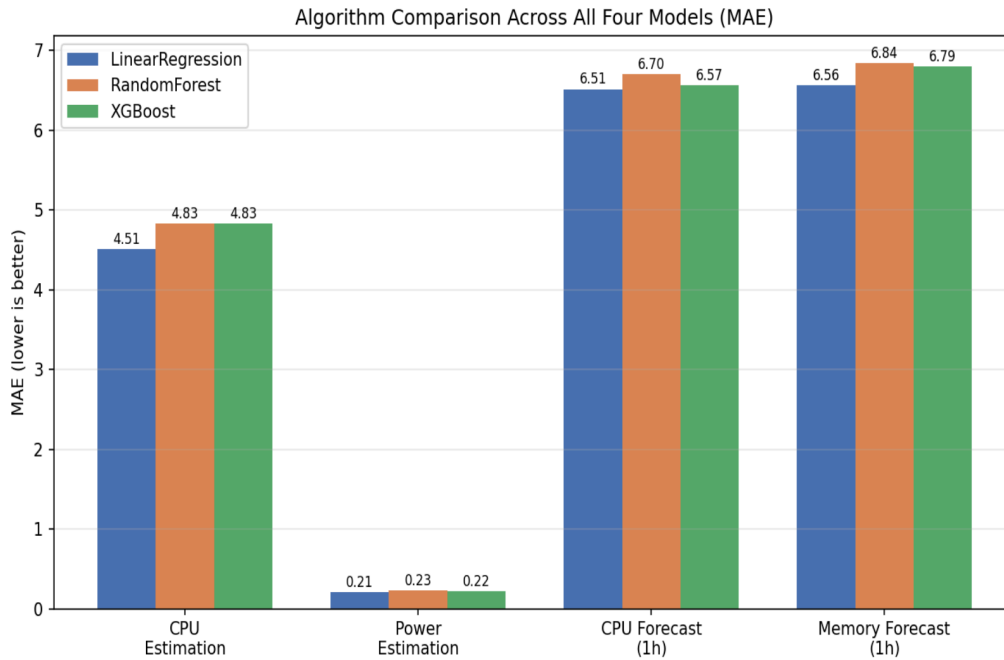


Fig 5.1. Comparison of evaluated algorithms across CPU estimation, power estimation, CPU forecasting and memory forecasting using MAE

The CPU model explains the large majority of variance in the target ($R^2 = 0.845$) using only workload and environmental features, with its own target correctly excluded to avoid leakage. The power model is noticeably weaker ($R^2 = 0.422$ at best) even with `cpu_utilization` included as an input. This is a genuine and expected finding rather than a bug and indicates that power draw in the real dataset depends on hardware and efficiency factors not fully captured by the available feature set. This asymmetry between the two targets is consistent with the general pattern reported in the wider literature on data-center power modelling (Section 2.4) and directly informed the decision to make the power model's prediction the primary basis for the what-if energy estimate, with a simpler CPU-ratio formula used only as a fallback when the model or the target's cooling telemetry is unavailable. Every number produced this way is explicitly labelled as an estimate rather than presented as a measurement. Figure 5.2 further illustrates this performance through the predicted-versus-actual plot, where the predictions show a close correspondence with the actual CPU utilization values across the held-out test set.

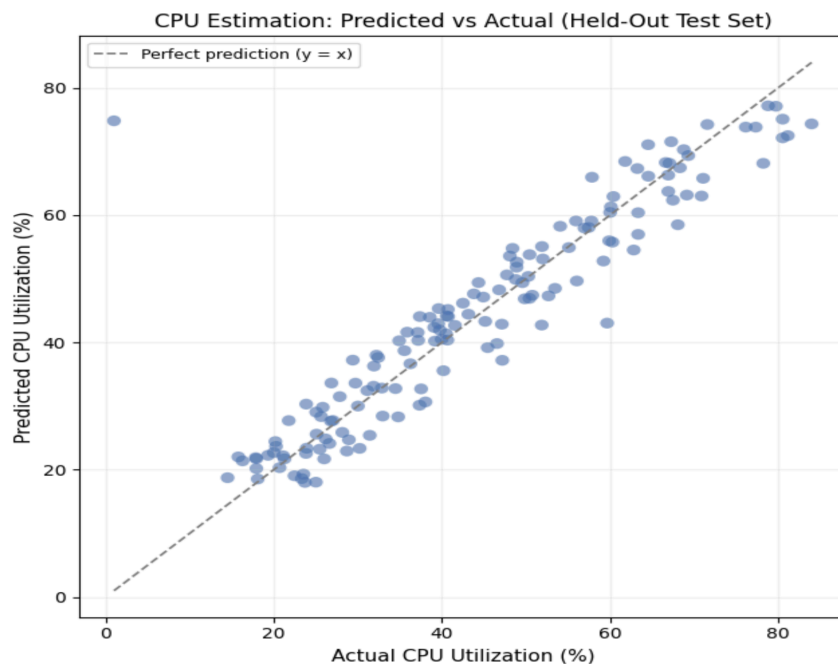


Fig. 5.2. CPU Estimation: Predicted vs Actual

5.3 WHAT-IF SIMULATION FOR SERVER CONSOLIDATION

Consider server S6 (Compute, workload bias 0.05, “very idle”), which is reliably flagged `idle_server` once its six-hour rolling average CPU falls below the Compute-type threshold of 17.3%. Requesting a what-if simulation for S6 causes the engine to enumerate every other Compute-type server not itself idle-flagged. In this fleet that means S1, S2 and S13. The engine computes each one's post-move CPU, memory and network as its own current load plus 90% of S6's current load. Each candidate is then checked against the four safety dimensions in Table 3.7. A candidate already running close to its own capacity or whose predicted post-move inlet temperature would exceed 30°C is excluded from `safe_now` regardless of headroom on other dimensions. Among the remaining candidates, the one with the most spare headroom after the move is chosen as the recommended target and its own fifteen-minute-ahead CPU and memory forecast is checked in exactly the same way before the recommendation is finalized as safe. The resulting recommendation card shows the chosen target, the full list of considered candidates with their pass/fail status on each dimension and an estimated facility-power impact derived from the trained power model, with the estimate's assumptions (which model was used and that it is a legitimate estimate rather than a measurement) stated explicitly alongside the number, consistent with the honesty principle followed throughout the system.

5.4 HONEST SUMMARY OF SYSTEM MATURITY

Consistent with the principle of never presenting an estimate with the same confidence as a measurement, the current system can be summarized in three honest categories.

5.4.1 GENUINELY REAL

The rule engine (pure threshold logic, no ML involved), PUE (a direct ratio of two independently simulated values), the offline ML validation results reported in Tables 5.1 and 5.2, the timestamp-based energy integration, and the four-dimension consolidation safety check.

5.4.2 LEGITIMATE ESTIMATES, CLEARLY LABELLED

Cost, carbon, water, and the power model's what-if energy prediction, which agrees with the simulator on direction and scale but will not match it exactly, since the two are independently built approximations of the same underlying quantity.

5.4.3 DELIBERATELY NOT BUILT

Criticality and SLA-aware gating, target storage-capacity and electrical/PDU-capacity safety checks, a verified (rather than operator-attested) confirmation mechanism for executed actions, the CPU model's designed hardware-health-check role (which needs environmental-fault injection in the simulator first), and trained six-hour/twenty-four-hour forecast models, for which there is not yet enough real telemetry history.

5.5 SYSTEM IMPLEMENTATION

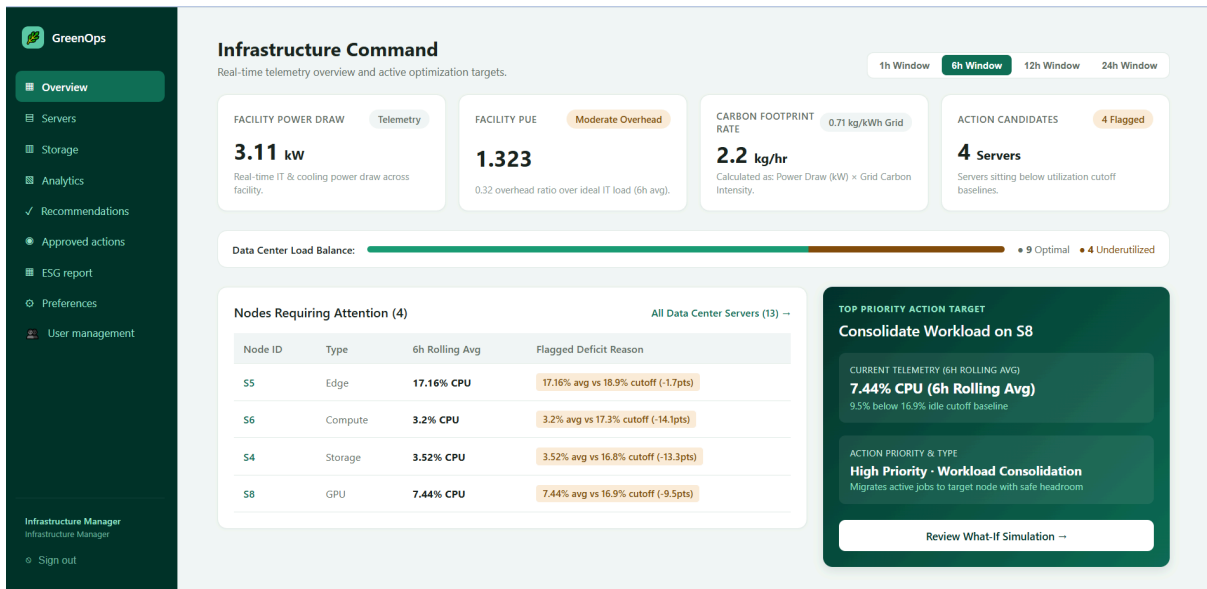


Fig. 5.3. Dashboard Overview Page

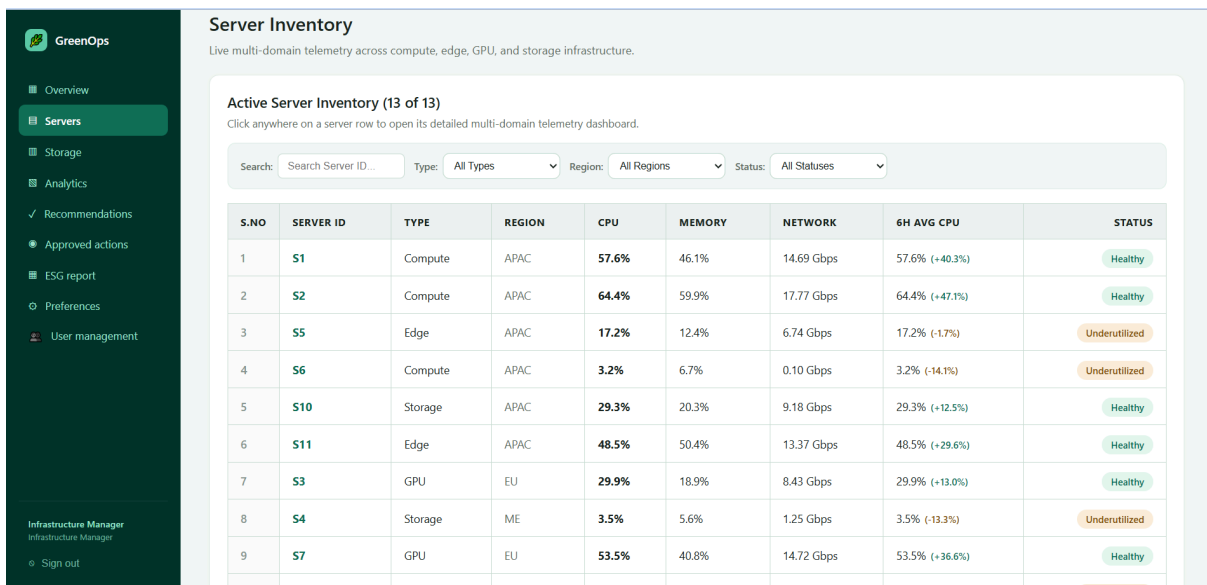


Fig. 5.4. Servers Details Page

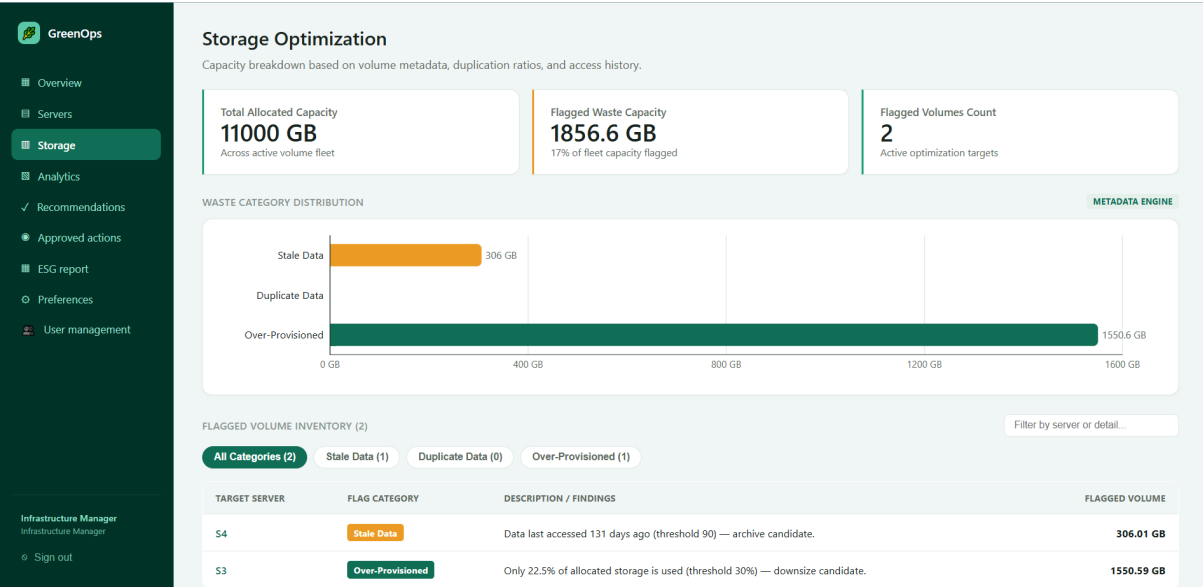


Fig. 5.5. Storage Details Page

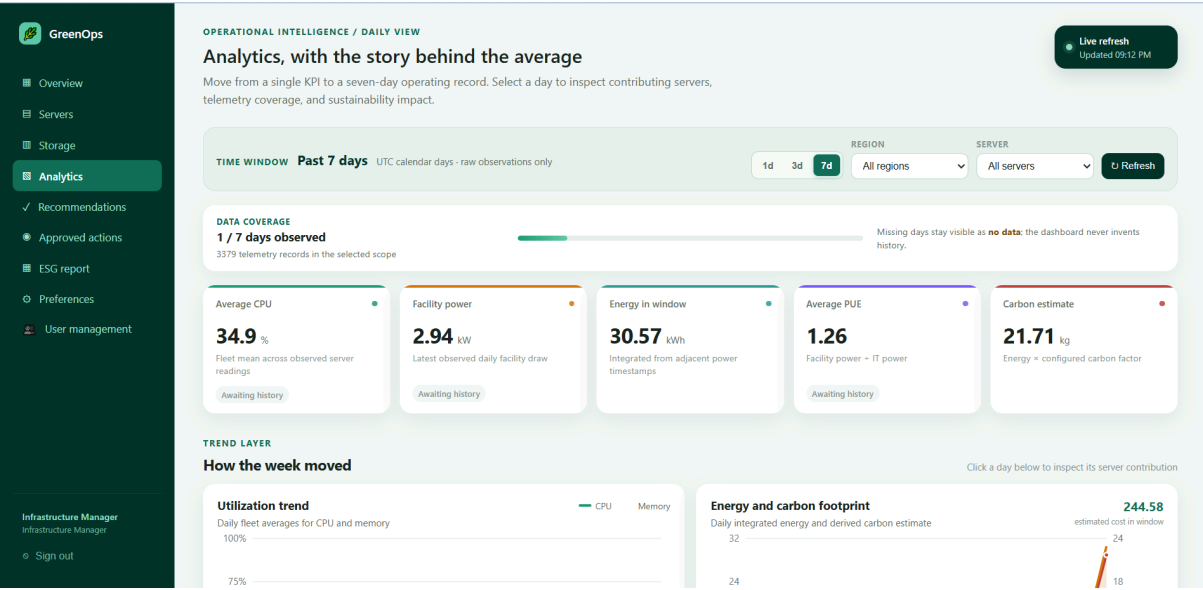


Fig. 5.6. Analytics Page

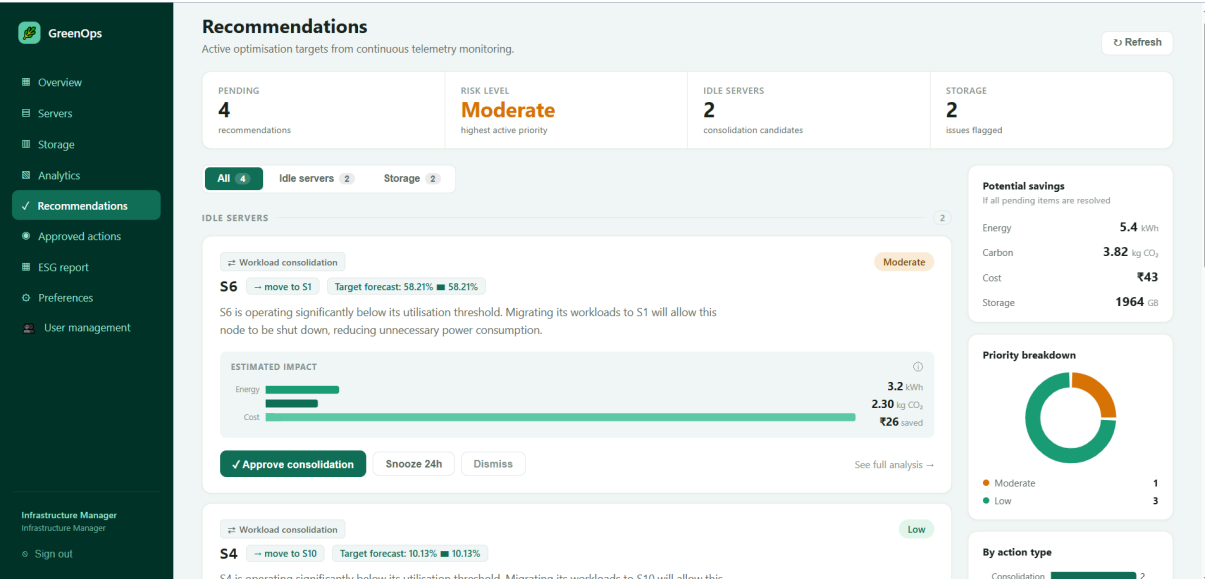


Fig. 5.7. Recommendations Page

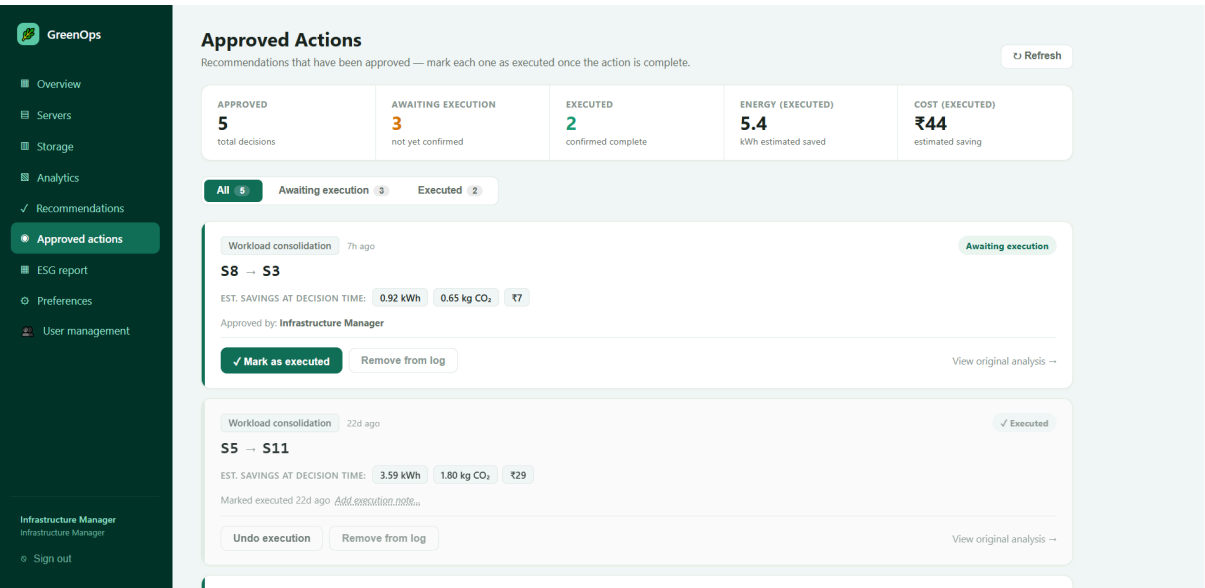


Fig. 5.8. Approved Actions Page

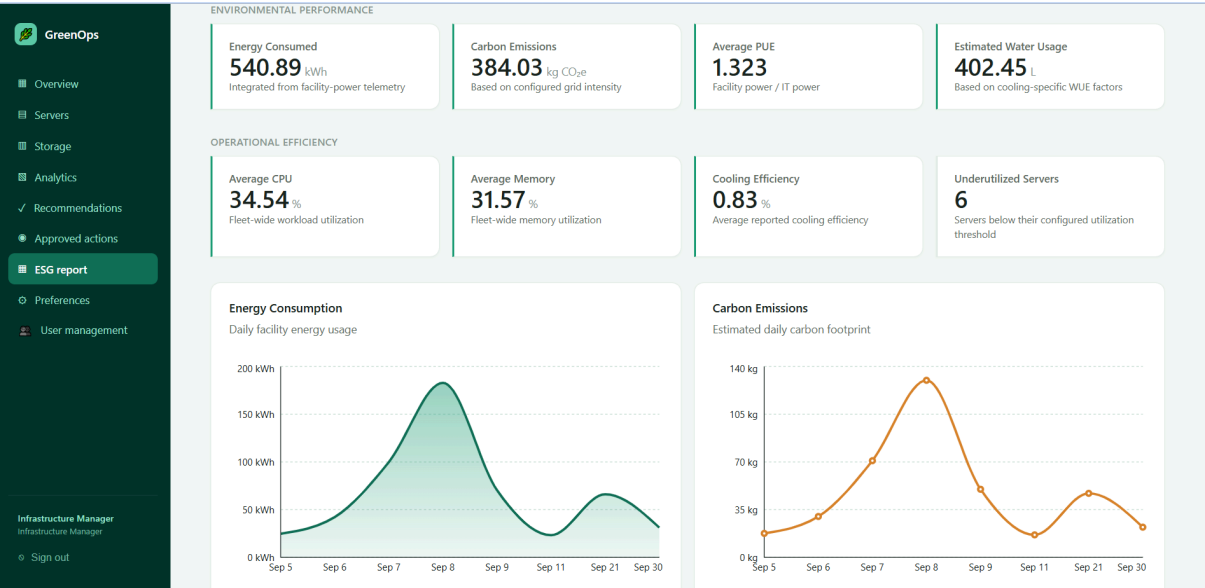


Fig. 5.9. ESG Reporting Page

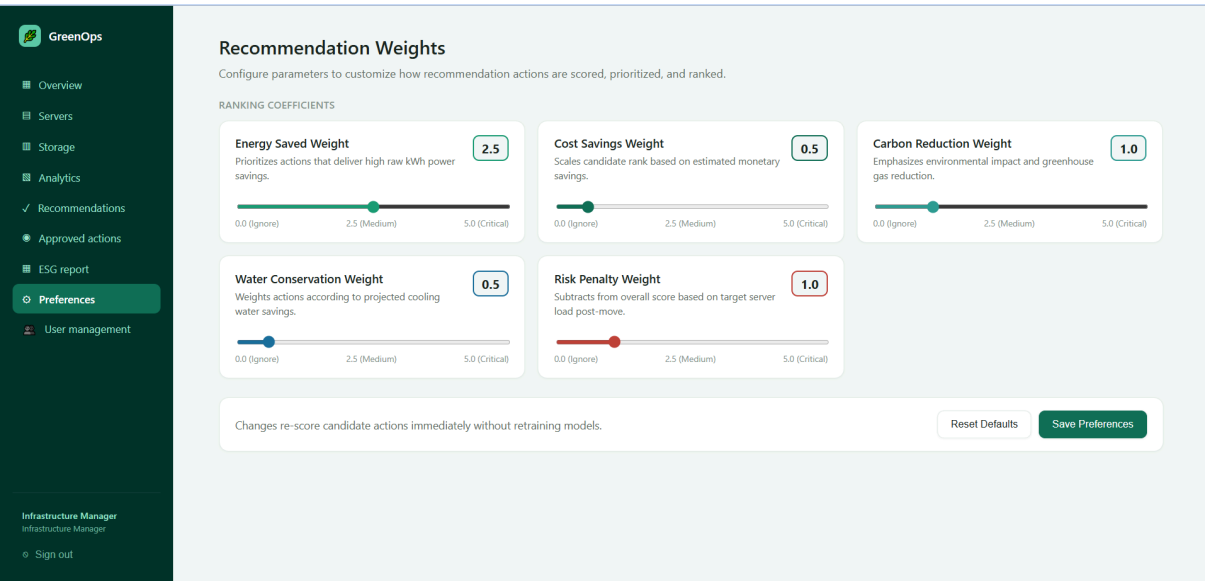


Fig. 5.10. Preferences Page

GreenOps

Overview

Servers

Storage

Analytics

Recommendations

Approved actions

ESG report

Preferences

User management

Infrastructure Manager

Infrastructure Manager

Sign out

User Management

Manage GreenOps accounts and control role-based access.

+ Add user

Users

Role permissions

TOTAL USERS

3

ACTIVE

3

INFRASTRUCTURE

1

SUSTAINABILITY

1

OPERATIONS

1

Active accounts

3 users

EMPLOYEE ID	NAME / EMAIL	ROLE	ADDED	ACTIONS
—	Sustainability Manager sustainability@greenops.local	Sustainability Manager	Sep 9, 2026	Edit Deactivate
—	Operations Engineer operations@greenops.local	Operations Engineer	Sep 9, 2026	Edit Deactivate
—	Infrastructure Manager infra@greenops.local	Infrastructure Manager	Sep 9, 2026	Edit Deactivate

Fig. 5.11. User Management Page

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

In this project, we have discussed the problem of inefficient resource utilization and sustainability in on-premise data centers and have developed GreenOps, a decision-support platform that would help in monitoring infrastructure resources and identifying possible optimization opportunities. The proposed system combines telemetry monitoring, sustainability calculation, workload forecasting and recommendation generation to provide a unified solution for managing underutilized servers in on-premise environments. It has been noted that the functioning of the system involves multiple stages, starting from telemetry collection and identification of underutilized resources, followed by workload prediction and what-if analysis before providing a recommendation to the operator. The system calculates important sustainability measures such as energy, carbon, water and cost, along with PUE and WUE, and uses statistically derived thresholds to identify inefficient resource usage. The implemented models achieved an R^2 value of 0.845 for CPU estimation and 0.422 for power estimation. The recommendation engine further allows different priorities to be assigned to sustainability and performance factors before suggesting suitable actions. The proposed system does not automatically execute any infrastructure changes, and the final decision remains with the human operator. Thus, GreenOps provides a transparent and priority-aware approach for supporting resource optimization and sustainability management in on-premise data center environments.

6.2 LIMITATIONS

The proposed system provides a working platform for resource monitoring and optimization in on-premise data centers, but its real-world implementation poses certain challenges. The telemetry used in the current system is simulated rather than collected from actual data-center hardware, and the cooling simulation does not currently represent independent equipment failures. The cost, carbon and water calculations are also based on configured values rather than live external data. The what-if analysis does not yet consider certain factors such as storage capacity, electrical capacity, workload criticality and SLA requirements. In addition, longer-term forecasting for twenty-four-hour periods has not been implemented due to the limited historical data available, and approved actions are currently confirmed by the operator rather than independently verified. Storage optimization and ESG reporting also remain as areas for further development.

6.3 FUTURE WORK

The proposed system can be further enhanced by replacing the simulated telemetry with real-time data from on-premise data-center infrastructure through sources such as Prometheus, hypervisor APIs, smart PDU and BMS systems. The cooling component can also be extended to support controlled environmental fault conditions, while live electricity tariffs, regional carbon-intensity information and actual water measurements can be incorporated to improve the sustainability calculations. The what-if analysis can be further developed by considering storage capacity, electrical capacity, workload criticality and SLA requirements before generating recommendations. With the availability of sufficient historical telemetry, a dedicated 24-hour workload forecasting model can also be developed to support longer-term resource optimization decisions. An independent verification mechanism can be

introduced to confirm approved actions and measure the actual savings achieved. The existing storage optimization and ESG reporting capabilities can also be further enhanced with additional analytics and reporting features, allowing GreenOps to provide a more comprehensive sustainability and resource-management solution for on-premise data centers.

APPENDIX 1 - KEY GENERATING FORMULAS

The following formulas, reproduced from the simulator implementation, generate every live telemetry reading in the system.

A1.1 SERVER_MONITOR.PY

$$workload = clamp(0.02, 0.98, gauss(bias, 0.08))$$

$$cpu = clamp(1, 99, workload \times 100 + uniform(-5, 5))$$

$$memory = clamp(5, 99, cpu \times 0.9 + uniform(-8, 8))$$

$$network = max(0.1, workload \times 25 + uniform(-2, 2))$$

A1.2 POWER_MONITOR.PY

$$variable_kw = 3.3 \times (0.85 \times cpu_fraction + 0.15 \times memory_fraction)$$

$$it_power_kw = max(0.3, 1.2 + variable_kw + uniform(-0.15, 0.15))$$

$$penalty_fraction = (0.99 - cooling_efficiency) \div (0.99 - 0.4)$$

$$overhead_ratio = max(1.05, 1.15 + 0.40 \times penalty_fraction + uniform(-0.03, 0.03))$$

$$facility_power_kw = it_power_kw \times overhead_ratio$$

A1.3 COOLING_MONITOR.PY

$$inlet_temperature_c = 19.0 + temp_rise_by_type[type] \times cpu_fraction + uniform(-0.8, 0.8)$$

$$cooling_efficiency = clamp(0.4, 0.99, 0.95 - efficiency_drop_by_type[type] \times cpu_fraction + uniform(-0.03, 0.03))$$

where $temp_rise_by_type = \{Air: 9.0, Hybrid: 5.0, Liquid: 2.5\}$ °C at 100% CPU, and $efficiency_drop_by_type = \{Air: 0.35, Hybrid: 0.20, Liquid: 0.10\}$.

A1.4 CONSOLIDATION POST-MOVE LOAD

$$post_move_cpu = target.current_cpu + source.current_cpu \times 0.9$$

$$post_move_memory = target.current_memory + source.current_memory \times 0.9$$

$$post_move_network = target.current_network + source.current_network \times 0.9$$

$$post_move_temp_c = 19.0 + temp_rise_by_type[target.type] \times \min(1.0, post_move_cpu / 100)$$

A1.5 RECOMMENDATION RANKING SCORE

$$score = \sum (weight_i \times normalized_impact_i) - risk_weight \times risk$$

APPENDIX 2 - DATABASE SCHEMA

The following eleven tables make up the shared PostgreSQL database described in Section 3. Nullable columns are marked accordingly. Timestamp columns default to the moment of insertion unless otherwise noted.

A2.1 USERS

Column	Type	Notes
id	Integer	Primary key
employee_id	String	Unique, nullable
name	String	Not null
email	String	Unique, not null
password_hash	String	infrastructure_manager, sustainability_manager or operations_engineer
is_active	Boolean	Default true
created_at	DateTime	Default current time

A2.2 SERVERS

Column	Type	Notes
id	Integer	Primary key
server_id	String	Unique, e.g. S1 through S13
server_type	String	Default Compute
cooling_type	String	Default Air
datacenter_region	String	Default APAC

A2.3 SERVER_TELEMETRY

Column	Type	Notes
id	Integer	Primary key
server_id	String	Indexed
cpu_utilization	Float	
memory_utilization	Float	
network_throughput_gbps	Float	
workload_intensity	Float	
timestamp	DateTime	Indexed

A2.4 POWER_TELEMETRY

Column	Type	Notes
id	Integer	Primary key
server_id	String	Indexed
it_power_kw	Float	
facility_power_kw	Float	
timestamp	DateTime	Indexed

A2.5 COOLING_TELEMETRY

Column	Type	Notes
id	Integer	Primary key
server_id	String	Indexed
inlet_temperature_c	Float	
cooling_efficiency	Float	
cooling_type	String	
timestamp	DateTime	Indexed

A2.6 STORAGE_TELEMETRY

Column	Type	Notes
id	Integer	Primary key
server_id	String	Indexed
total_storage_gb	Float	
used_storage_gb	Float	
duplicate_data_gb	Float	
last_accessed_days_ago	Integer	
storage_type	String	
timestamp	DateTime	Indexed

A2.7 FORECASTS

Column	Type	Notes
id	Integer	Primary key
server_id	String	Indexed
horizon_minutes	Integer	60,360 or 1440 for the dashboard tier
measured_cpu	Float	
predicted_cpu	Float	Nullable when there is not enough history
method	String	e.g. LinearRegression or historical_average
model_version	String	Nullable
validation_mae	Float	Nullable
validation_rmse	Float	Nullable
validation_r2	Float	Nullable
source_observations	Integer	Default 0
generated_at	DateTime	Indexed

A2.8 FLAGS

Column	Type	Notes
id	Integer	Primary key
server_id	String	Indexed
flag_type	String	idle_server, stale_data, duplicate_data or overprovisioned
severity	String	low, medium or high
detail	String	Human-readable explanation
metric_value	Float	e.g. average CPU or GB reclaimable
created_at	DateTime	Indexed
resolved	Integer	0 active, 1 resolved

A2.9 RECOMMENDATIONS

Column	Type	Notes
id	Integer	Primary key
flag_id	Integer	Indexed, unique, not null
server_id	String	Indexed, not null
recommendation_type	String	consolidate, archive, deduplicate or rightsize
priority	String	high, medium or low
explanation	String	Not null
status	String	pending, accepted, rejected or snoozed
created_at	DateTime	Indexed
decided_at	DateTime	Nullable
snoozed_until	DateTime	Nullable

A2.10 OPERATOR_ACTIONS

Column	Type	Notes
id	Integer	Primary key
recommendation_id	Integer	Indexed, not null
server_id	String	Indexed, not null

Column	Type	Notes
action	String	consolidate, rightsize, snooze or do_nothing
notes	String	Nullable
estimated_energy_saving_kwh	Float	Nullable, snapshot at decision time
estimated_carbon_reduction_kg	Float	Nullable, snapshot at decision time
estimated_cost_saving	Float	Nullable, snapshot at decision time
estimated_storage_reclaimed_gb	Float	Nullable, snapshot at decision time
target_server_id	String	Nullable, the chosen consolidation target if any
executed_at	DateTime	Nullable, null means approved but not yet executed
execution_note	String	Nullable
decided_by_user_id	Integer	Foreign key to users.id, nullable
decided_by_name	String	Nullable, snapshot at decision time
decided_by_role	String	Nullable, snapshot at decision time
created_at	DateTime	Indexed

A2.11 PREFERENCES

Column	Type	Notes
id	Integer	Primary key, single row by convention
energy_weight	Float	Default 1.0
cost_weight	Float	Default 1.0
carbon_weight	Float	Default 1.0
water_weight	Float	Default 0.5
risk_weight	Float	Default 1.0
updated_at	DateTime	Updated on every write

REFERENCES

- [1] Beloglazov, A., Abawajy, J., and Buyya, R., “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012.
- [2] Subramanian, C., Vasan, A., and Sivasubramaniam, A., “Reducing data center power with server consolidation: Approximation and evaluation,” in *Proc. 17th Int. Conf. High Performance Computing (HiPC)*, 2010.
- [3] Lovász, G., Niedermeier, F., and de Meer, H., “Performance tradeoffs of energy-aware virtual machine consolidation,” *Cluster Computing*, vol. 16, no. 3, pp. 481–496, 2013.
- [4] Uddin, M. *et al.*, “Server consolidation: A technique to enhance cloud data center power efficiency and overall cost of ownership,” *International Journal of Distributed Sensor Networks*, vol. 17, no. 3, 2021.
- [5] Bawa, J., Chahal, K. K., and Kaur, K., “Improving cloud resource management: An ensemble learning approach for workload prediction,” *The Journal of Supercomputing*, vol. 81, no. 10, Art. no. 1138, 2025.
- [6] Deva, M. R. R., and Jain, N. K., “Utilizing Azure Automated Machine Learning and XGBoost for predicting cloud resource utilization in enterprise environments,” in *Proc. 2025 Int. Conf. Networks and Cryptology (NETCRYPT)*, 2025, pp. 535–540.
- [7] Khan, T., Tian, W., Ilager, S., and Buyya, R., “Workload forecasting and energy state estimation in cloud data centres: ML-centric approach,” *Future Generation Computer Systems*, vol. 128, pp. 320–332, 2022.

- [8] Radovanovic, A. *et al.*, “Carbon-aware computing for datacenters,” arXiv:2106.11750, 2021.
- [9] Zhan, X. *et al.*, “Data center cooling system optimization using offline reinforcement learning,” arXiv:2501.15085, 2025.
- [10] Alex, M., Ojo, S. O., and Awuor, F. M., “Carbon-aware, energy-efficient, and SLA-compliant virtual machine placement in cloud data centers using deep Q-networks and agglomerative clustering,” *Computers*, vol. 14, no. 7, Art. no. 280, 2025.
- [11] Grochulska-Salak, M. *et al.*, “Green data centres: Sustainable solutions with green energy and green–blue infrastructure,” *Energies*, vol. 18, no. 24, Art. no. 6592, 2025.
- [12] European Commission, “Corporate sustainability reporting,” Financial Markets, Company Reporting and Auditing.
- [13] Tripathi, V. *et al.*, “Scaling India’s data centre ecosystem: Stakeholder perspectives on infrastructure, energy, and resilience,” Council on Energy, Environment and Water and SYSTEMIQ, 2026.
- [14] Tao, Y., and Gao, P., “Global data center expansion and human health: A call for empirical research,” *Eco-Environment & Health*, vol. 4, no. 3, Art. no. 100157, 2025.
- [15] Gorey, J., “Data drain: The land and water impacts of the AI boom,” *Land Lines Magazine*, Lincoln Institute of Land Policy, Oct. 17, 2025.
- [16] Vaccaro, V., Tagliabue, L. C., and Aldinucci, M., “Sustainable data centers: Advancing energy efficiency and resource optimization,” in *Proc. 33rd Euromicro Int. Conf. Parallel, Distributed and Network-Based Processing (PDP)*, Turin, Italy, 2025, pp. 486–493.

- [17] Ahmed, A., Brown, D., and Gegov, A., “Dynamic resource allocation through workload prediction for energy efficient computing,” in *Advances in Computational Intelligence Systems*, 2017, pp. 35–44.
- [18] “Green AI Data Center Telemetry” dataset, Kaggle.
- [19] React, “React: The library for web and native user interfaces,” React Documentation.
- [20] Vite, “Vite: Next Generation Frontend Tooling,” Vite Documentation.
- [21] Recharts, “Recharts: A composable charting library built on React components,” Recharts Documentation.
- [22] Scaphandre, “Scaphandre: A metrology agent dedicated to electrical power consumption,” Scaphandre Documentation.
- [23] OpenCost, “OpenCost: Open source cost monitoring for cloud native environments,” OpenCost Documentation.